Intractability,NP,NP-Complete

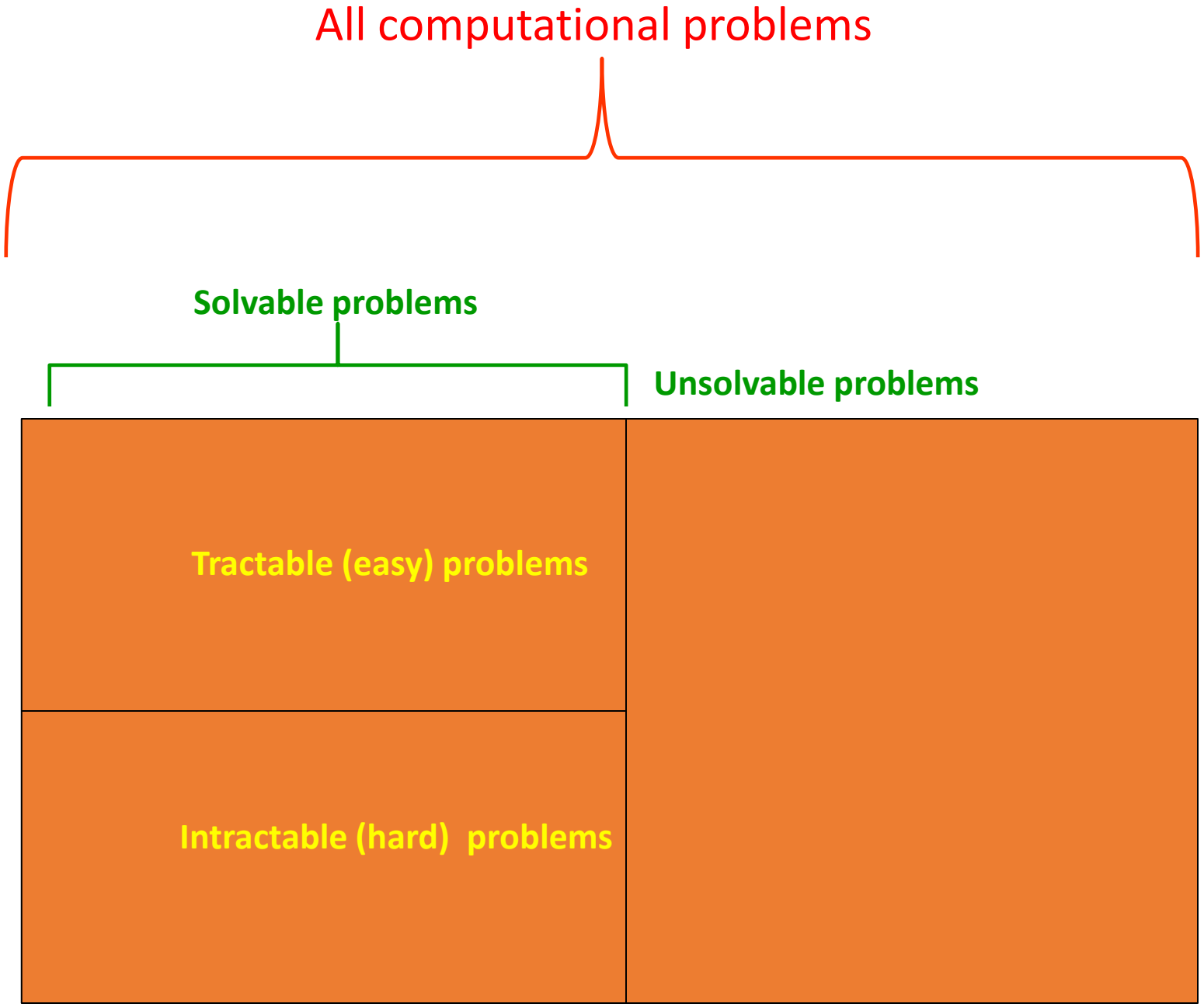
Can all computational problems be solved by a computer?

The one million dollar question:

P = NP ?

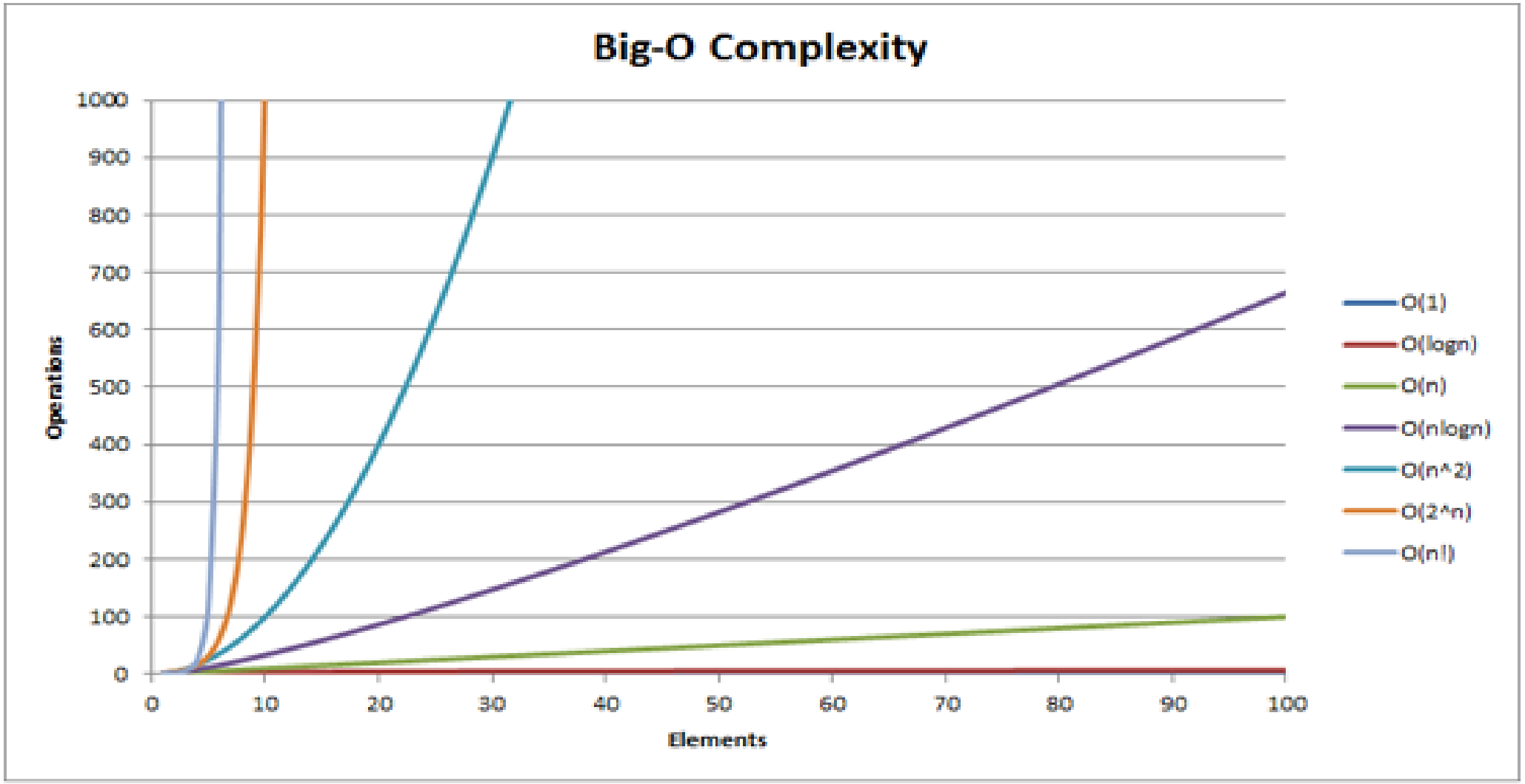
http://www.claymath.org/millennium-problems

Prof.Dr.M. C.Okur


All computational problems
Solvable problems
Unsolvable problems
Tractable (easy) problems
Intractable (hard) problems

How bad is exponential complexity ?

- Fibonacci example – The recursive fib cannot even compute fib(30)



Tractability

- Some problems are *intractable*:
  as they grow large, we are unable to solve them in reasonable time
- What constitutes reasonable time?
  - Standard definition: *polynomial time*
  - On an input of size n the worst-case running time is $O(n^k)$ for some constant k
  - Polynomial time: $O(1)$, $O(\log n)$, $O(n)$, $O(n \lg n)$, $O(n^2)$, $O(n^3)$, …
  - Not polynomial time: $O(2^n)$, $O(n!)$, $O(n^n)$…

Polynomial-Time Algorithms

- What kind of problems are solvable in polynomial time?
  - Many algorithms we've studied provide polynomial-time solutions to some problems
- Are all problems solvable in polynomial time?
  - No. Unfortunatelly,many important problems are not solvable by any computer, no matter how much time is given
- Most important problems that do not yield polynomial-time algorithms are either optimization or decision problems.

Decision VS. Optimization Problems

- Decision problem: Solving the problem by giving an answer "YES" or "NO"
- Optimization problem: Solving the problem by finding the optimal solution.
- Examples:
  - **SHORTEST-PATH (optimization)**
    - Given G, u,v, find a path from u to v with fewest edges.
  - **PATH (decision)**
    - Given G, u,v, and k, is there a path from u to v consisting of at most k edges?

Decision vs Optimization Problems

- **Some problems are decidable, but *intractable*:**
  As they grow large, we are unable to solve them in reasonable time.
- An optimization problem tries to find an optimal solution
- A decision problem tries to answer a yes/no question
- Many problems will have decision and optimization versions
  - Eg: Traveling salesman problem
    - optimization: find Hamiltonian cycle of minimum weight
    - decision: is there a Hamiltonian cycle of weight $\leq$ k ?

(Hamiltonian cycle is a cycle passing(visiting) every vertex of a graph exactly once)

Example : Finding Hamiltonian Cycles

- Can we find a polynomial-time algorithm that solves a given optimization or decision problem?
- Determining whether a directed graph has a Hamiltonian cycle does not have a polynomial time algorithm.

Optimization problem:

Given a weighted graph, find a Hamiltonian cycle with the smallest total weight.

Decision Problem:

- Guess a permutation of all vertices.
- Check whether this permutation gives a cycle.

  If yes, then algorithm halts.

Example :Finding Minimum Spanning Trees

- Optimization Problem :Given a weighted graph G, find a spanning tree with minimum total weight.
- Decision problem: Given a weighted graph G and a positive integer k, does G contain a spanning tree with total weight $\leq$ k ?
  - Guess a spanning tree T.
  - Check whether the total weight of T $\leq$ k.

The Complexity Class P

- The set P is defined as the set of all problems that can be solved in *polynomial worse case time*
  - Also known as the *polynomial time* complexity class

  →Refers to problems that have solution *algorithms* whose running times is $O(n^k)$ for some k
- Examples of problems in P: Searching, sorting, shortest paths, tree traversals, string matching…

Deterministic vs Nondeterministic

- Deterministic algorithm : For a given particular input, the algorithm will always produce the same output going through the same states.

→ The algorithm always travels a single path from input to output

- Non-deterministic algorithm:For the same input, the algorithm may produce different outputs in different runs.

Insanity: Doing the same thing over and over again and expecting different results.
Albert Einstein.

Deterministic vs Nondeterministic

- A *determinstic* computer is what we know as a computer,
- A *nondeterministic* computer is one that can "guess" the right answer or solution
  - Think of a nondeterministic computer as a parallel machine that can freely spawn (Create) *an infinite number* of processes at the same time.
- A N*ondeterministic* computer can execute non deterministic algorithms to solve a specific class of decision problems in polynomial time.
- Solution here means producing a yes or no answer.

The Complexity Class NP

NP: the class of decision problems that are solvable in polynomial time with a nondeterministic algorithm

- Thus *NP* can also be thought of as the class of problems
  - whose solutions can be verified in polynomial time
- *NP* stands for "Nondeterministic Polynomial-time"

NP does not stand for nonpolynomial –time !

Polynomial Time Verification

- NP means verifiable in polynomial time. Verifiable?

→If we are somehow given a 'certificate' of a solution we can verify the correctness of it in polynomial time.

Examples:

- PATH problem: Given <G,*u*,*v*,*k*>, is there a path from *u* to *v* with at most *k* edges?

→Given a path *p* from *u* to *v*, verify whether the length of *p* is at most *k* ? This can be done in polynomial time.

- Sorting problem: We are given a permutation of the elements.

→We Can verify in linear time if a candidate ordering of the elements is sorted or not.

Polynomial Time Verification

- Hamiltonian cycle problem is in the NP class: *Why?*
- Given an order of the *n* distinct vertices $(v_1, v_2, \ldots, v_n)$, we can test if (v_i, v_{i+1}) is an edge in G for $i=1, 2, \ldots, n-1$ and (v_n, v_1) is an edge in G , in time $O(n)$.
- Given a candidate path,we can test in linear time if it is a Hamiltonian cycle – just check if all vertices are visited exactly once in the candidate path

→*Complexity of the test :*Polynomial in the input size.

P is a Subset of NP

- Nondeterministic algorithms don't exist in the real life – it is a purely theoretical idea invented to understand how hard a computational problem could be
- Question : Are any problems in P also in NP?
  - YES! *All* problems in P are also in NP
    - Notation: $P \subseteq NP$, why ?
    - If you can solve a problem in polynomial time, you can definitely verify a solution in polynomial time
    - Example : Sorting.

    →Verification can not take more time than the sorting algorithm itself!

Converse Question: Is NP a Subset of P ?

- Question: Are all problems in NP also in P?
  - Is NP $\subseteq$ P ?
  - Unfortunately , no one knows!
  - Also , no one can tell NP $\nsubseteq$ P

  This is a million dollars question!
- Now it is time to meet "NP-Complete" problems.

Polynomial Reducibility

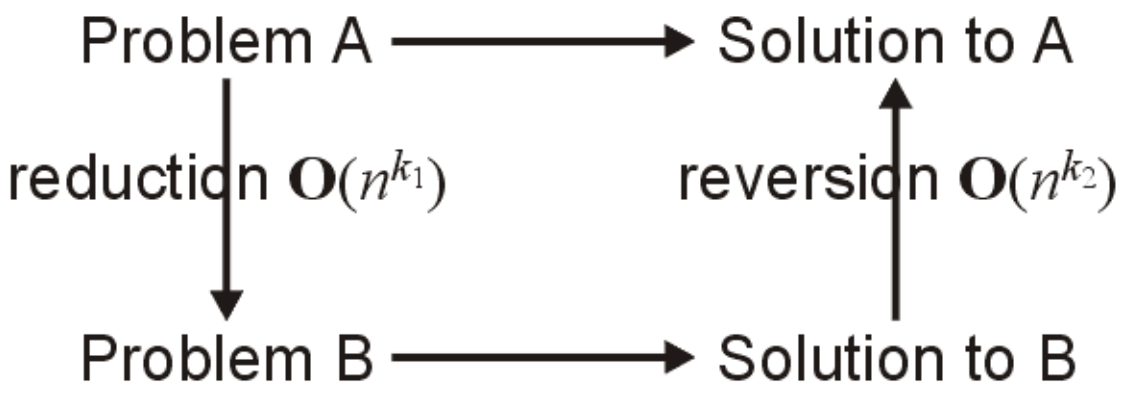
- A problem A can be reduced to another problem B if any instance of A can be "easily rephrased" as an instance of B.
- Consider an instance α of decision problem A and an instance β of decision problem B
- A reduction from A to B is a transformation with the following properties:
  - The transformation takes polynomial time
  - The answer is the same for both problems:

  The answer for α is YES if and only if the answer for β is YES.

→The solution to B provides a solution to the instance of A

Polynomial reduction

Graphically, we may think of the following image:



To solve Problem A, we:

- Reduce it to Problem B in polynomial time
- Find the solution to Problem B
- Revert this solution back into a solution for Problem A

Polynomial Reduction: Examples

1. Multiply two *n* digit decimal numbers by a binary multiplier:
   - Reduction: convert the two numbers into binary numbers
   - Multiply the two binary numbers
   - Reversion: convert the solution back into a decimal number

   → Both the reduction and the reversion run in O(*n*) time

2. Does a list have a duplicate element?
   - Reduction: Sort the list
   - Simpler problem: Does a sorted list have a duplicate element?
   - Reversion: Check and return `true` or `false` as answer

   → Both the reduction and the reversion run in polynomial time.

The Class NP-Complete

A problem x is NP-complete if

1. $x \in NP$ and
2. every problem $y \in NP$ can be reduced to x.

→If any NP-compete problem is polynomial -time solvable, then P=NP !

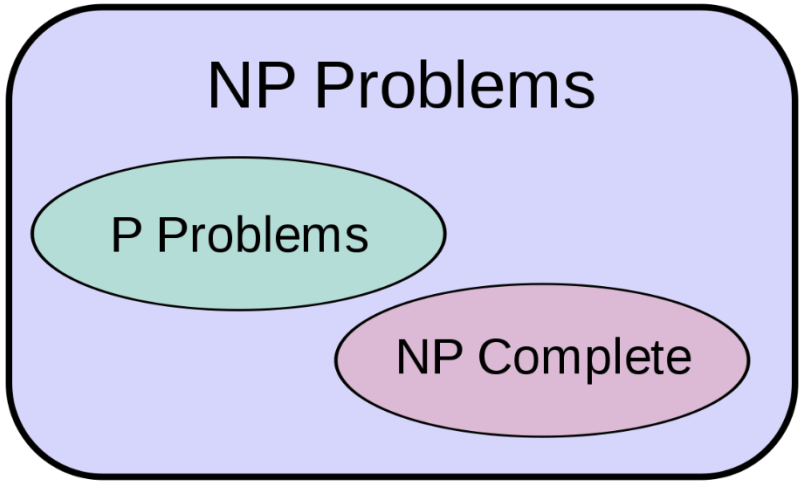
Or conversely : if any problem in NP is not poly-time solvable, then no NP-complete problem is poly-time solvable.

In other words :If any one of the NP-Complete problems was to be solved quickly, then all *NP* problems could be solved quickly!

The Class-NP Complete

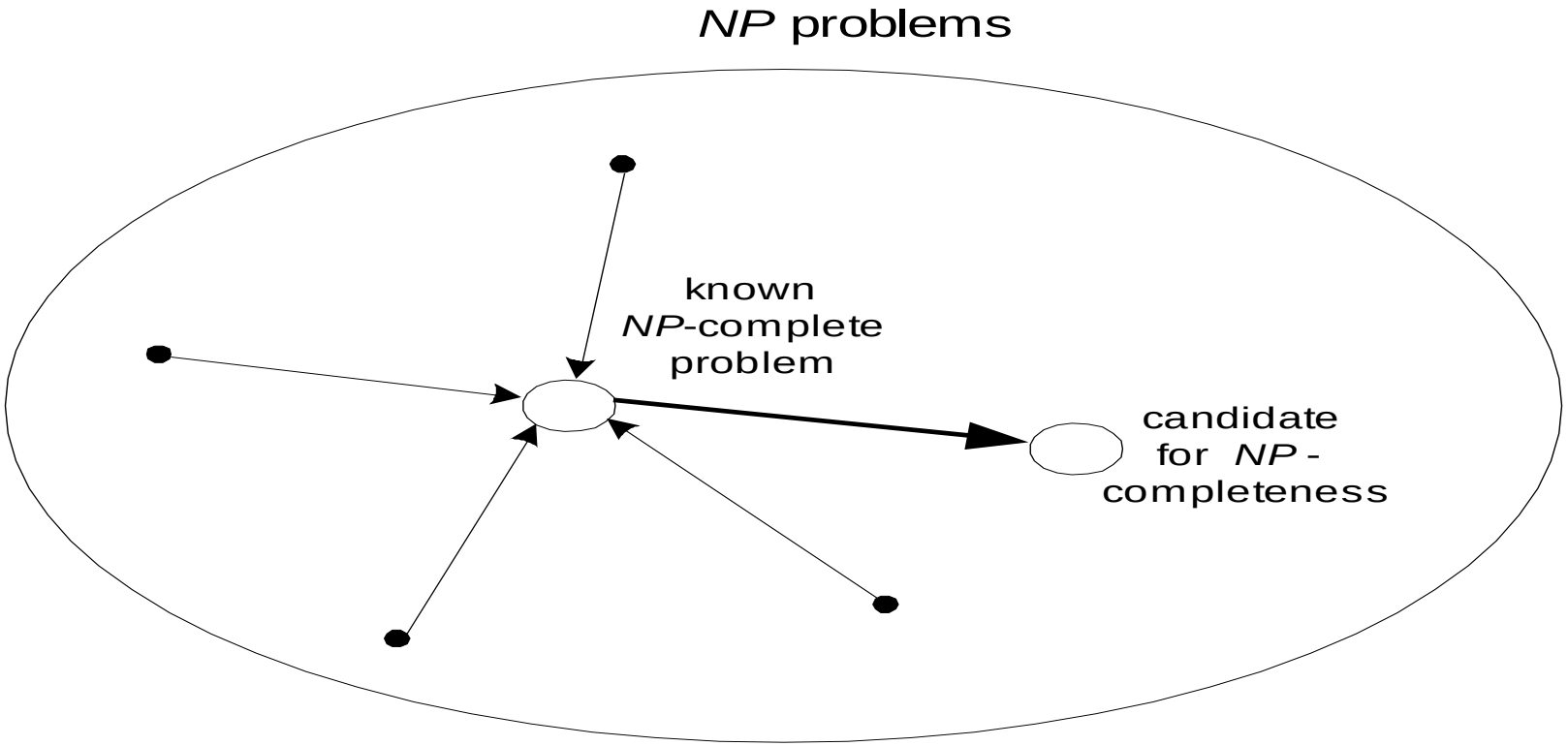
- NP = all the problems for which once you have the answer, it is easy to verify it
- NP Complete : All problems in this set can be reduced to each other.

→ If a problem in this class has an efficient algorithm (in P), then every other NP Complete problem will have an efficient algorithm!



NP-Completeness and Reduction

NP-complete problems can be obtained through polynomial-time reductions from a known *NP*-complete problem

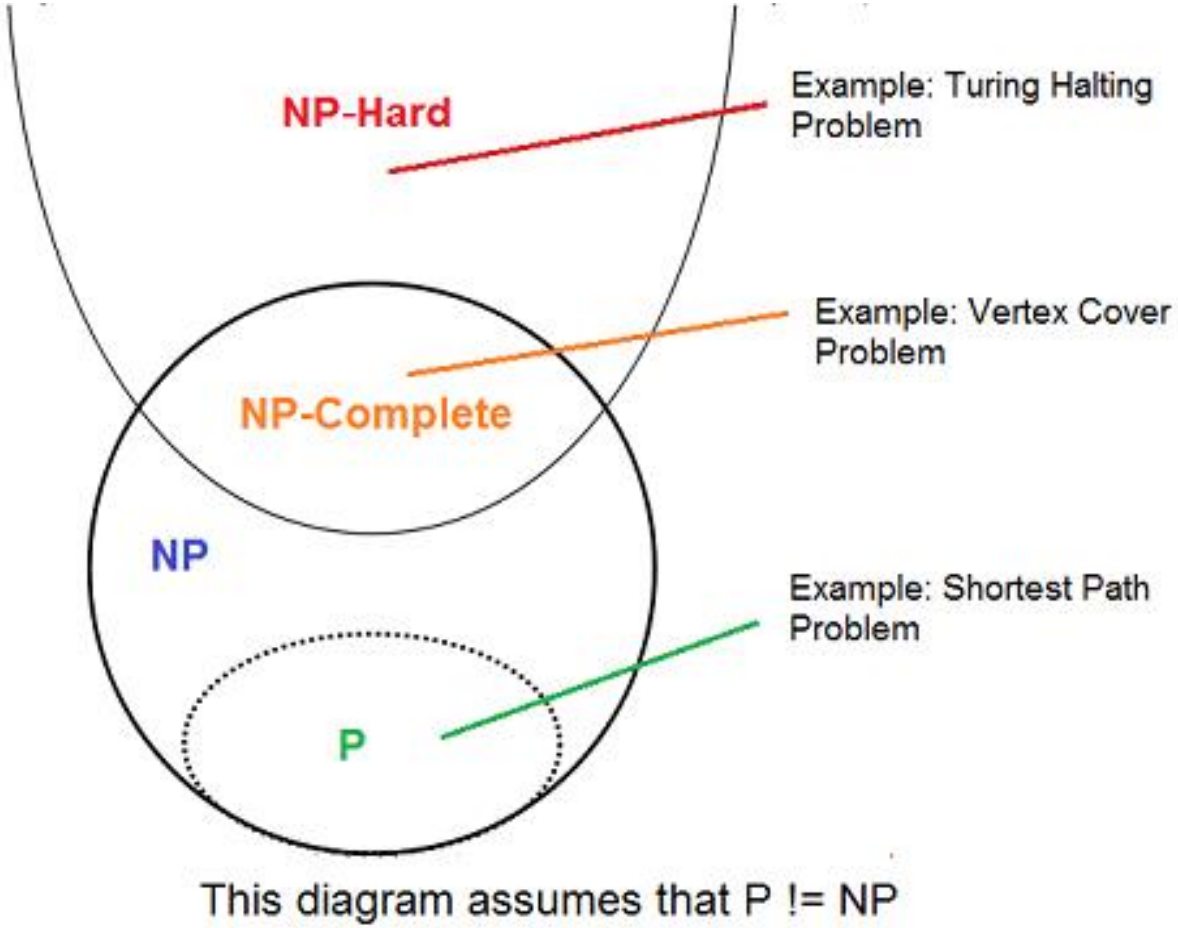


Examples: TSP, knapsack, partition, graph-coloring and hundreds of other problems of combinatorial nature

Why NP-Complete is Important ?

- Most but not all researchers believe that $P \neq NP$
- Many algorithms in cryptography depend on the fact that no known solutions exist for NP complete problems.
- The whole internet is also based on the assumption that $P \neq NP$
- In fact , our financial systems , many technologies in daily life are built upon this assumption.
- It would definitely have a significant impact on computing ,science and technology… if a solution to just one of the NP Complete problems were found.

Problem Classes : General Picture



(Given a graph, find the minimum sized vertex set that covers all edges)

Note : NP-Hard class includes the problems that are at least as hard as the NP-complete problems.
We are not going to consider this class here.

First NP-Complete Problem : Boolean Satisfiability

- SAT : One of the first problems to be proved NP-Complete was Boolean *satisfiability* (SAT):
  - Given a Boolean expression of *n* variables in conjunctive normal form(Products of sums), can we assign truth values such that the expression evaluates to TRUE?

  Ex: $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (\neg x_5)$....

  Solution:Determine if there is an assignment of values to the variables that makes the formula true (=1) for m clauses.
  - Seems simple enough, but no known deterministic polynomial time algorithm exists for the solution!

Solving Boolean-satisfiability Problem

- Intuitive solution:
  - for each possible assignment, check whether it generates 1.
  - suppose the number of input variables is k, then the total possible assignments are 2^k. So the running time is $O(2^k)$.

  → Running time is not polynomial.

Solving Boolean-satisfiability Problem

- Running time is not polynomial , but is it polynomial-time verifiable?
- In other words, is it NP?
  - If someone gives you a solution how long does it take to verify it?
  - Make one pass through the formula and check : Takes poly time.

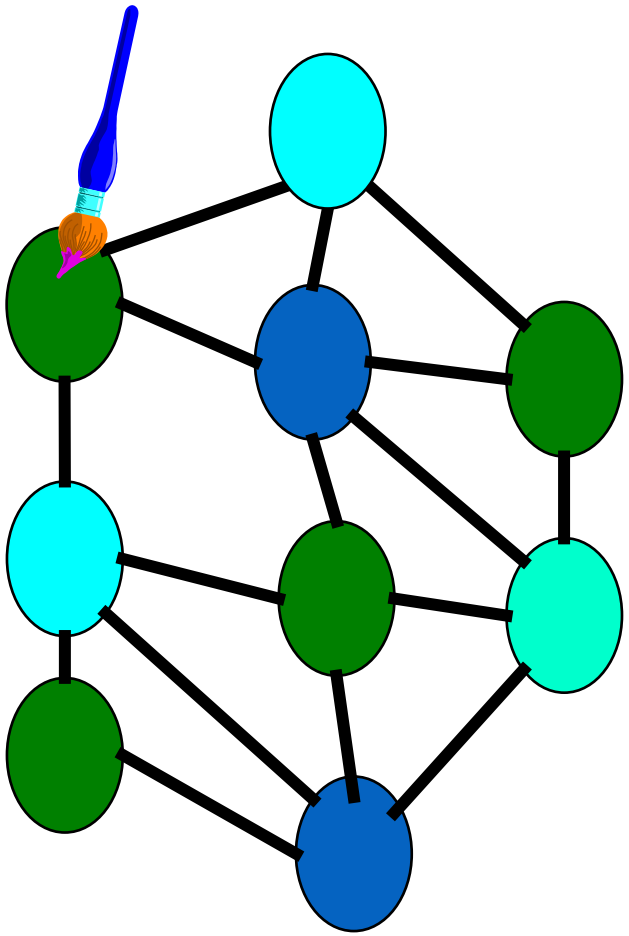
→Verifiable in polynomial time!

Satisfiability is an NP problem !

NP-Completeness and Solution

- *Remember what steps do we have to take to prove a problem P is NP-Complete?*
  - Pick a known NP-Complete problem Q
  - Reduce Q to P
    - Describe a transformation that maps instances of Q to instances of P, such that "yes" for P → "yes" for Q
    - Prove that the transformation works
    - Prove that verification runs in polynomial time
      → $P \in NP$

K-COLORING Problem is NP Complete



Color each node : No adjacent nodes can have the same color.

→Nodes with edges between them must have different colors.

- Problem : Given a graph G and a k, find a way to color G with k colors.

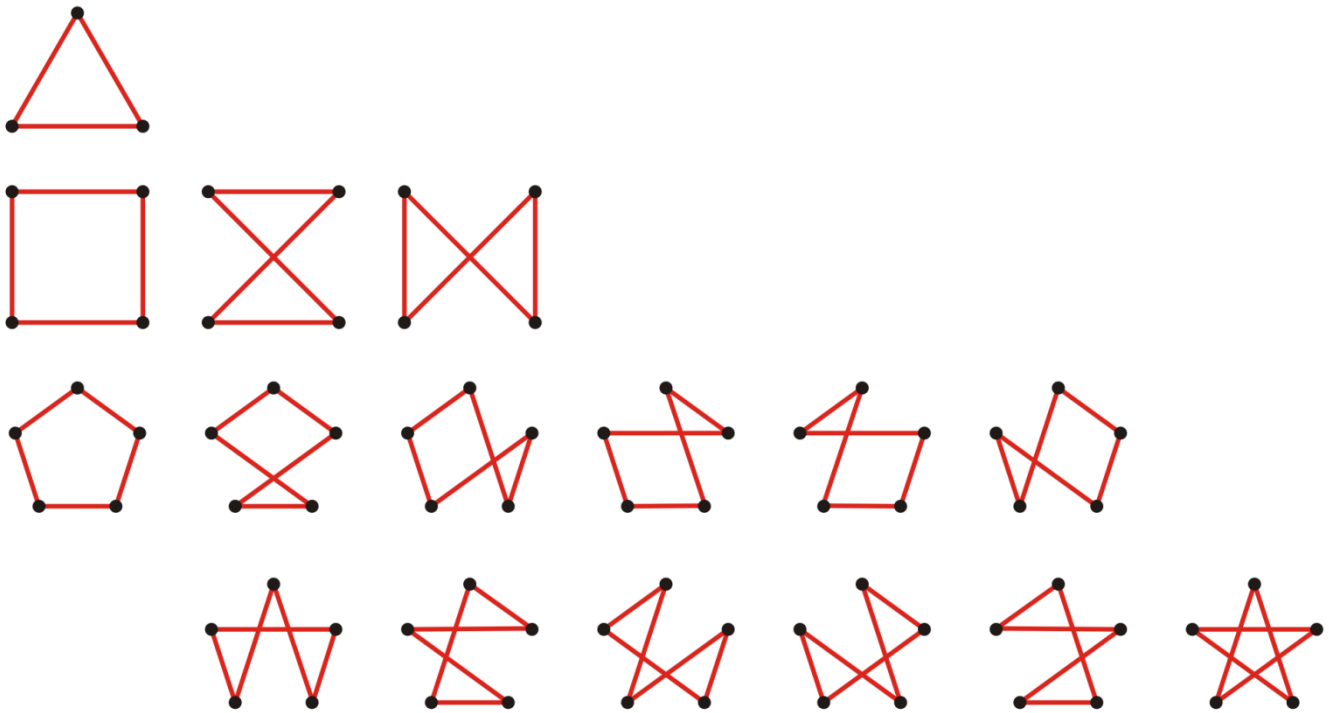
Travelling Salesman Problem

- The well-known *traveling salesman problem*:
  - Complete graph with cost c(*i,j*) from city *i* to city *j*
  - Is there a simple cycle over cities with cost < *k* ?
- *How can we prove the TSP is NP-Complete?*
- Ans: Prove that TSP $\in$ NP; reduce the undirected Hamiltonian cycle(HC) problem to TSP
  - Reduction: need to show that if we can solve TSP we can solve Hamiltonian cycle problem

  → If the answer to TSP is YES, then HC is YES. Likewise if TSP is NO, then HC is NO.

NP Completeness of TSP

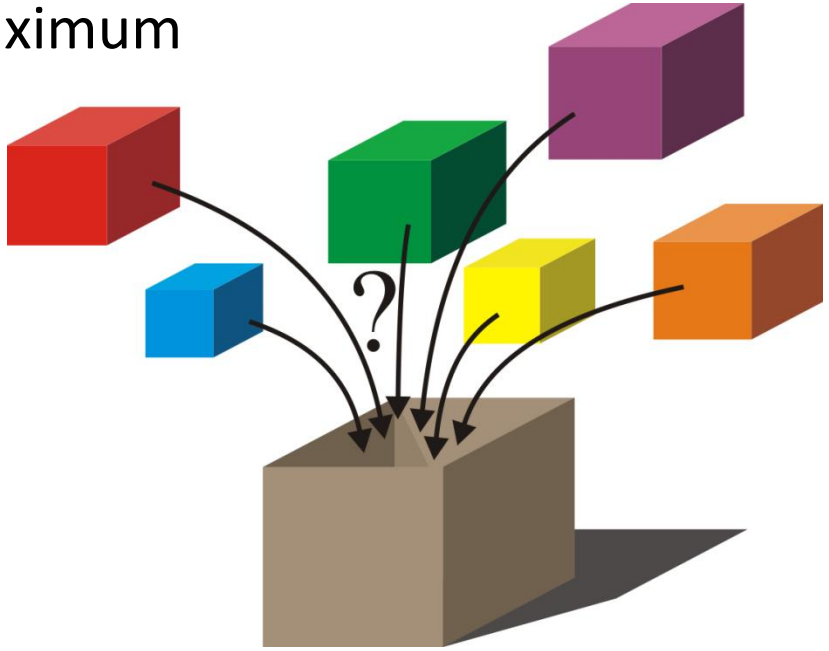
- From the description of the TSP, it can be shown that there are (n-1)!/2 different Hamiltonian paths and there exists one TSP path for each HC(Reduction).
- Trying all these paths would be $\Theta(n!)$. Ex: Possible HC or TSP paths for n=3,4,5:



NP Version of Knapsack Problem

The knapsack decision problem :

- Suppose a container can hold a maximum weight W
- Given a number of items with specified weights and values, is there some combination of items that has a total value greater than or equal to V without exceeding the maximum weight W?
- Note that we are not trying to find the maximum value,but trying to find a subset of items satisfying some constraint.



Summary: P,NP,NP-Complete

- *What do we mean when we say a problem is in P?*
  - Ans : A solution can be found in polynomial time
- *What do we mean when we say a problem is in NP?*
  - Ans: A solution can be verified in polynomial time
- *What is the relation between P and NP?*
  - Ans: P $\subseteq$ NP, but no one knows whether P = NP or $P \neq NP$
- NP-Complete:
  - Problems that are decidable in polynomial time by a nondeterministic algorithm
- Biggest Open Question:
  - Does P = NP?

IF P=NP…

- Transportation of all forms will be scheduled optimally to move people and goods around quicker and cheaper.
- Manufacturers can improve their production to increase speed and create less waste.
- Learning becomes easy :For any problem, we simply find and implement the smallest program by brute force, consistent with the data.
- Near perfect vision recognition, language comprehension and translation and all other learning tasks become trivial.
- We will also have much better predictions of weather and earthquakes and other natural phenomena.

Epilogue:

Most computer scientists and mathematicians believe that $P \neq NP$

and the whole internet is also based on this assumption !